Escape Barbershop — documentation

Five documents, each answering a different question.

Document	Answers
Proposal	What we are building, for whom, and why it is one system rather than three
Design	How it is put together: the data model, the rules, the decisions and their reasons
MVP	What "finished enough to use" means, and what is deliberately left out
Roadmap	What is done, what is next, and what each phase needs from you
Testing	The two suites, what they cover, and how to add to them

Start with the roadmap — it carries the ticked boxes and the open questions.

Where things stand

Everything on the roadmap is built, tested and deployed except the phone apps.

The management system runs the shop as it always did — visits, the drawer, expenses, advances, payroll, the warehouse, the reports. On top of it now: customers book from their phone and are rung up as one visit; the queue and the diary; barbers with their own day, earnings and reviews; reminders, confirmations and review requests as phone notifications; a loyalty card, referrals and birthdays; reviews; insight screens; and customer groups the owner can message.

Since then: products sold at the till, a waiting list that turns a cancellation back into a booking, deposits on the expensive work, a screen for the shop wall, barbers asking for their own time off, their own portfolios, and a no-show policy that asks for a deposit rather than barring anybody. Most recently, the conversation around the chair: a customer keeps photos of the cut they want where the barber sees them, and the barber reports what happened during a cut where only the owner reads it.

What is left is **phase 4** — the same app published to Google Play and the App Store — which waits on the shop's own developer accounts. The roadmap says what those cost.

For whoever works on this next

- `npm run dev` — the app on http://localhost:3000
- `npm test` — the rules (booking, payments, loyalty, reviews, sessions), no database, seconds
- `npm run test:all` — those plus the API against a real Postgres (needs Docker)
- `npm run test:preview` — a walk through the customer app against the demo shop

- `npm run docs:pdf` — rebuild the PDFs in `docs/pdf`
- `docker compose up --build` — the whole thing with its own database and demo data

The rules live in `src/lib` as plain functions with tests beside them — `booking.ts`, `payments.ts`, `finance.ts`, `session-policy.ts`, `loyalty.ts`, `reviews.ts`, `referrals.ts`, `birthdays.ts`, `segments.ts`, `insights.ts`. They are the part worth reading first: every screen is a view onto those, and the server enforces them independently of what any screen allows.

The two apps are dressed from one set of tokens at the top of `src/app/globals.css`. Nothing in the customer or barber app names a colour of its own, which is what lets the whole thing turn from night to day by swapping that block.
